



ITES
RENÉ DESCARTES

Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Rodrigo Alejandro Barbosa Vidal

Fecha:
03/07/26

Introducción

En el desarrollo de software es común que varias personas trabajen al mismo tiempo en un mismo proyecto. Sin una buena organización, pueden surgir problemas como pérdida de información y conflictos entre archivos. Por ello, el control de versiones es una herramienta esencial, ya que permite registrar los cambios, identificar quién los realizó y facilitar el trabajo en equipo. En este documento se analiza el caso del proyecto SmartLibrary y se propone una estrategia de trabajo utilizando Git.

Parte 1. Análisis del Problema

1. ¿Cuáles fueron los principales errores cometidos por el equipo durante el desarrollo?

El equipo trabajó compartiendo una carpeta en la nube en lugar de usar un sistema de control de versiones. Esto provocó que se sobrescribieran archivos, se perdieran cambios y aparecieran errores al juntar el trabajo de todos.

2. ¿Por qué trabajar únicamente compartiendo carpetas genera tantos problemas?

Cada desarrollador trabaja en una copia diferente del proyecto. Al subir una nueva versión, puede borrar el trabajo de otro compañero. Además, no hay un historial para saber qué cambios se hicieron o cómo recuperarlos.

3. ¿Qué consecuencias podría tener esta forma de trabajo en proyectos grandes?

Podría causar muchos errores, pérdida de información, retrasos en el proyecto y dificultades para coordinar el trabajo del equipo, aumentando también el tiempo y los costos de desarrollo.

Parte 2. Importancia del Control de Versiones

¿Qué es un sistema de control de versiones?

Es una herramienta que guarda y organiza los cambios realizados en un proyecto, permitiendo llevar un historial de todas las modificaciones.

¿Cuál es su principal objetivo?

Su objetivo es facilitar el trabajo en equipo, controlar los cambios en el código y evitar que se pierda información.

¿Qué ventajas ofrece frente al manejo tradicional de archivos?

Guarda un historial de cambios, Permite recuperar versiones anteriores, Muestra quién realizó cada modificación, Facilita el trabajo de varios desarrolladores al mismo tiempo., Reduce la pérdida de información y los conflictos entre archivos.

Parte 3. Propuesta de Organización

¿Qué rama utilizarías para la versión estable del sistema?

La rama main, ya que solo debe contener las versiones estables y revisadas del proyecto.

¿Qué rama utilizarías para integrar el trabajo del equipo?

La rama develop, donde se reúnen los cambios de todos los desarrolladores antes de pasar a main.

¿Cada desarrollador debería trabajar directamente sobre la rama principal? Explica tu respuesta.

No, cada desarrollador debe trabajar en una rama propia (por ejemplo, feature/autenticacion o feature/usuarios) para evitar conflictos y no afectar la versión estable del proyecto.

¿Cuándo deberían integrarse los cambios al proyecto?

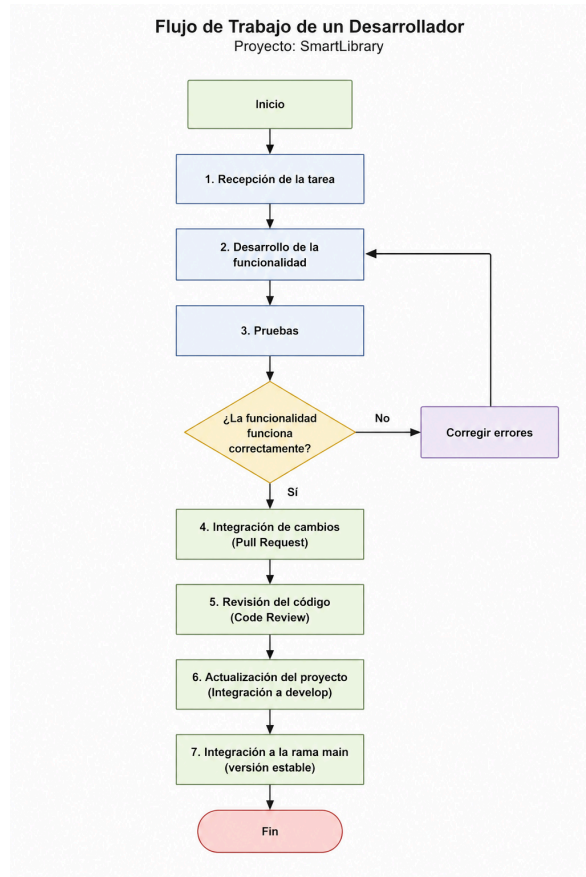
Cuando la funcionalidad esté terminada, probada y funcione correctamente. Después se integra primero a develop y, cuando todo esté listo, a main.

¿Cuándo deberían integrarse los cambios al proyecto?

Los cambios deberían integrarse únicamente cuando la funcionalidad esté terminada, haya sido probada correctamente y no presente conflictos con el resto del proyecto. Antes de incorporarse a la rama principal es recomendable realizar una revisión del código para asegurar su calidad.

Parte 4. Flujo de Trabajo

Diagrama de flujo propuesto:



Parte 5. Análisis de Situaciones

1. ¿Qué ocurriría si dos desarrolladores modifican el mismo archivo simultáneamente?

Git detectará un conflicto y los desarrolladores deberán revisar y combinar los cambios antes de continuar.

2. ¿Por qué es importante conservar un historial de cambios en un proyecto?

Porque permite saber qué cambios se hicieron, quién los realizó y recuperar versiones anteriores si ocurre un error.

3. ¿Qué beneficios aporta trabajar mediante ramas independientes?

Permite que cada desarrollador trabaje sin afectar el código principal y facilita la integración de cambios.

4. ¿Cómo ayuda el control de versiones cuando aparece un error después de una actualización?

Ayuda a encontrar el cambio que causó el error y permite volver a una versión anterior si es necesario.

5. ¿Por qué el control de versiones es indispensable en proyectos desarrollados por varios programadores?

Porque organiza el trabajo en equipo, evita la pérdida de información y facilita el control de los cambios realizados.

Conclusión

con esta actividad aprendí la importancia del control de versiones en el desarrollo de software, entendí que trabajar solo compartiendo archivos puede causar pérdida de información, conflictos y errores, git se ha convertido en un estándar porque permite organizar mejor el código, trabajar en equipo de forma segura y mantener un historial de todos los cambios realizados, una buena organización del proyecto ayuda a reducir errores, mejorar la colaboración entre los desarrolladores y lograr un desarrollo más eficiente.